

Estructuras de control

Tema 2

ÍNDICE

01 Estructura secuencial

02 Estructuras condicionales

03 Estructuras repetitivas

04 Estructuras perniciosas

MOTIVACIÓN

- Conocer el funcionamiento de las estructuras condicionales
- Conocer el funcionamiento de las estructuras repetitivas
- Saber diferenciar y escoger las estructuras de control adecuadas para resolver problemas de programación



01

Estructura secuencial

Conceptos básicos

- Las estructuras de control de un lenguaje de programación se refieren al orden en que las instrucciones de un algoritmo se ejecutarán.

- El orden de ejecución de las sentencias o instrucciones determinan el flujo de control.

Estructuras básicas de control



Secuencial

Instrucciones consecutivas

Selección

Algunos grupos de instrucciones se ejecutan (o no)

Repetición

Grupos de instrucciones se ejecutan varias veces

Estructura secuencial

Las sentencias se ejecutan sucesivamente, en el orden en que aparecen. No existen “saltos” o bifurcaciones.

```c
/* Calculo de Raices de una ecuacion de grado 2. */
#include <stdio.h>
#include <math.h>

int main() {
    double a, b, c, r1, r2;

    printf("\nIntroduce coeficiente de 2o grado: ");
    scanf("%lf", &a);
    printf("\nIntroduce coeficiente de 1er grado: ");
    scanf("%lf", &b);
    printf("\nIntroduce coeficiente independiente: ");
    scanf("%lf", &c);

    r1 = (-b + sqrt(b*b - 4*a*c)) / (2*a);
    r2 = (-b - sqrt(b*b - 4*a*c)) / (2*a);

    printf("Las raices son %lf y %lf\n", r1, r2);

    return 0;
}
```

Estructura secuencial

- Si bien el programa anterior es correcto, no es capaz de manejar situaciones excepcionales.
- Por ejemplo, ¿qué pasa en la siguiente sentencia si a = 0?

```
r1 = ( -b + sqrt( b*b-4*a*c ) ) / (2*a);
```

- En este caso obtendríamos un error de ejecución por intentar hacer una división por cero.
- Solución: estructuras condicionales
  - ...pero antes....

Diagrama de flujo

- Es una representación gráfica de un algoritmo
- Se construyen a partir de símbolos especiales que se conectan mediante flechas que indican el flujo de ejecución.

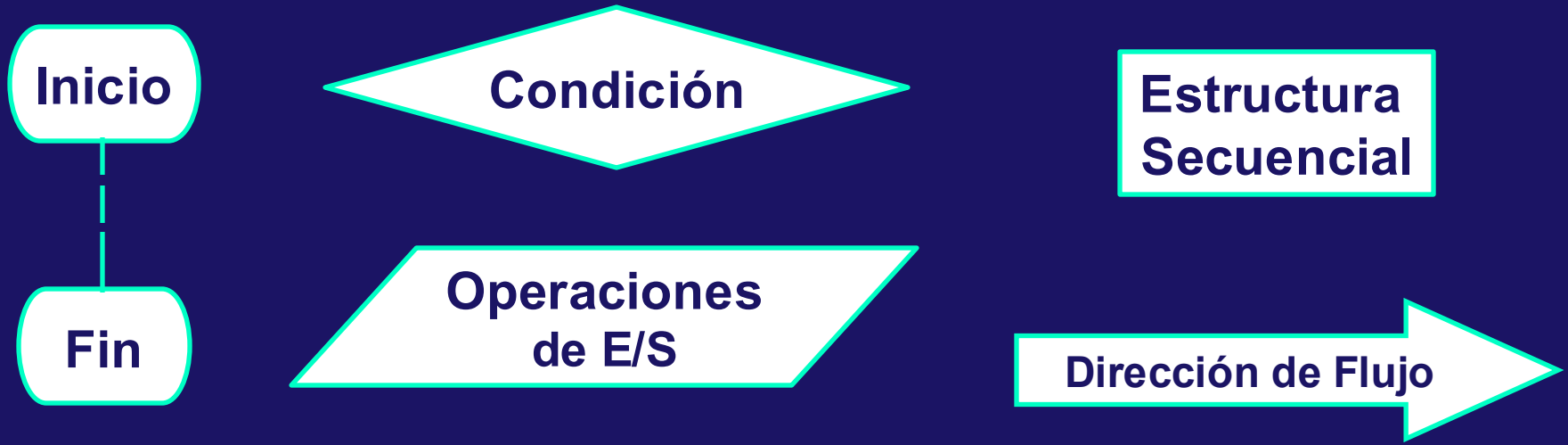
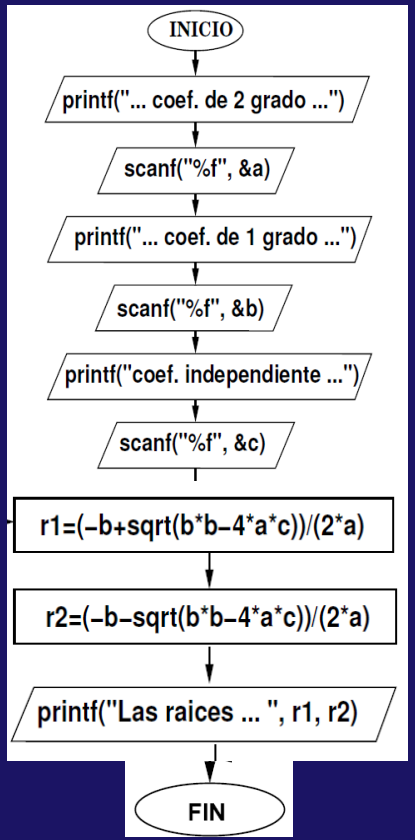


Diagrama cálculo raíces



02

Estructuras condicionales

Estructura de selección

- También recibe el nombre de estructura de selección
- Permite elegir entre diferentes cursos de acción en función de condiciones:

Si la nota del examen es mayor o igual que 5 → mostrar “Aprobado”

- Si la condición es verdadera,
  - Se ejecuta la sentencia mostrar, y luego el programa continuaría en la sentencia siguiente al “*Si*”
- Si la condición es falsa,
  - La sentencia mostrar se ignora y el programa continúa

Estructura condicional simple

La instrucción en pseudo código

Si la nota del examen es mayor o igual que 5
mostrar *"Aprobado"*

Se traduce a C mediante una sentencia condicional simple:

```c
if ( nota >= 5 ){
    printf("Aprobado");
}
```

La forma general es:

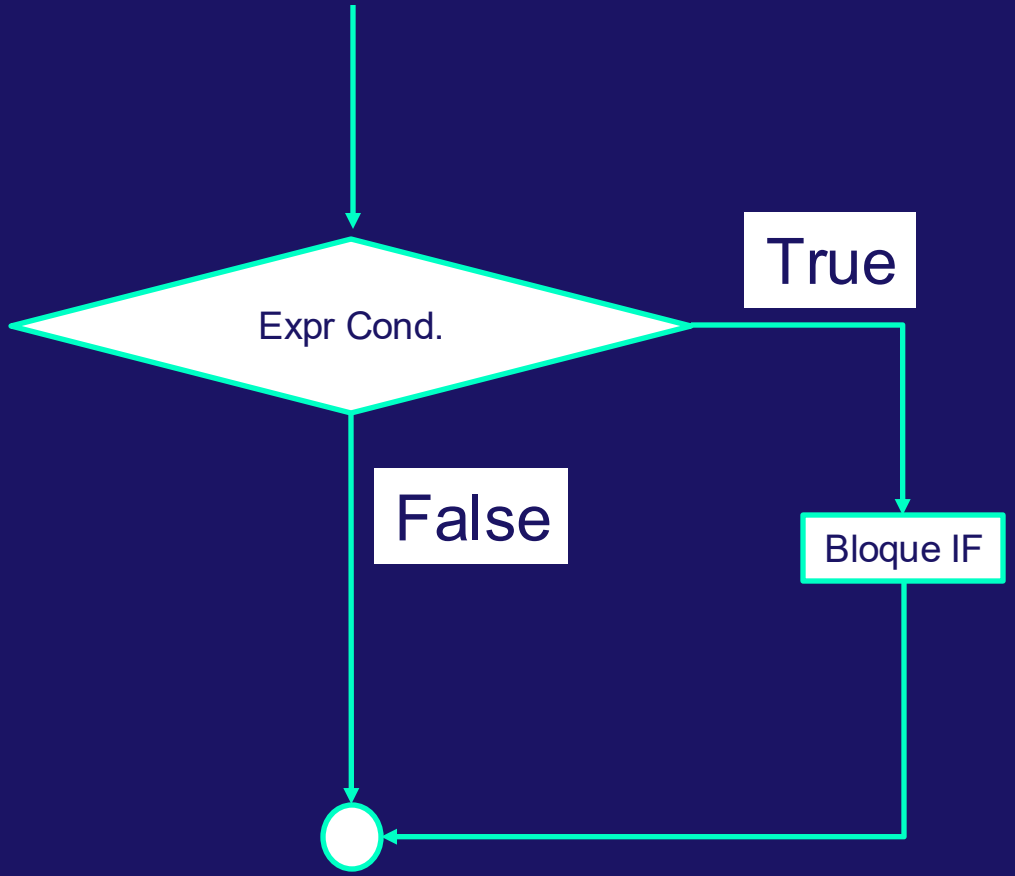
if (<condicion>)
<bloque if>;

Una expresión lógica

Una sentencia o conjunto de sentencias (encerradas entre {})

Estructura condicional simple

- El diagrama de flujo asociado es como se indica



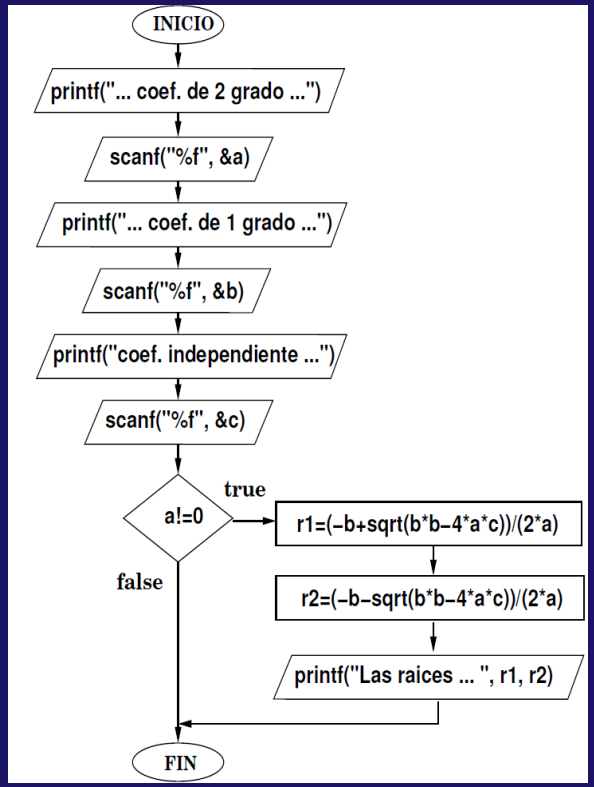
Cálculo de raíces

```c
/* Calculo de Raices: control de la variable "a" */
#include <stdio.h>
#include <math.h>

int main(){
    double a, b, c, r1, r2;
    printf("\nIntroduce coeficiente de 2o grado: ");
    scanf("%lf", &a);
    printf("\nIntroduce coeficiente de 1er grado: ");
    scanf("%lf", &b);
    printf("\nIntroduce coeficiente independiente: ");
    scanf("%lf", &c);

    if (a != 0){
        r1 = ( -b + sqrt(b*b - 4*a*c) ) / (2*a);
        r2 = ( -b - sqrt(b*b - 4*a*c) ) / (2*a);
        printf("Las raices son %lf y %lf\n", r1, r2);
    }
    /*if(a == 0){
        printf("El coeficiente de 2o grado no puede ser cero.\n");
    } */
    return 0;
}
```

Diagrama cálculo raíces



Ejemplo

```c
#include <stdio.h>

int main(){
    int dato1, dato2;
    char opcion;

    printf("Introduce el primer operando: ");
    scanf("%d", &dato1);
    printf("Introduce el segundo operando: ");
    scanf("%d", &dato2);
    printf("Selecciona (S) sumar, (R) restar: ");
    scanf(" %c", &opcion);  /* Espacio antes de %c para ignorar saltos de línea */

    if (opcion == 'S'){
        printf("Suma = %d\n", dato1 + dato2);
    }
    if (opcion == 'R'){
        printf("Resta = %d\n", dato1 - dato2);
    }
    if ((opcion != 'R') && (opcion != 'S')){
        printf("Ninguna operación\n");
    }

    return 0;
}
```

Condiciones compuestas

- En muchos casos tendremos que utilizar condiciones compuestas:
  - Compruebe si un número real está dentro de un intervalo cerrado [inferior, superior].

```
if (dato >= inferior && dato <= superior)
```

  - Lea el valor de la edad y salario de una persona. Súbale el salario un 4% si tiene más de 45 años y además gana menos de 1300 euros.

```
if (edad >= 45 && salario < 1300)
```

Ejercicios (condicionales)

- Saber si un número es positivo
- Obtener si una pareja de números son ambos negativos
- Obtener si un número es par
- Saber si un carácter está en mayúsculas
- Definir si un número de teléfono (representado como un entero) pertenece la provincia de Granada
- Tomar tres números y saber si el 3º es la suma de los dos 1ºs
- Calcular si un año es bisiesto
- Dados tres valores (día, mes, año) saber si la fecha es correcta
- Dadas las coordenadas de un rectángulo (esquina superior izquierda e inferior derecha) y un punto, comprobar si está dentro del anterior rectángulo.

Estructura condicional doble

- Permite elegir entre 2 cursos de acción diferentes en función del valor de verdad de una expresión lógica.

Si la nota del examen es mayor o igual que 5
mostrar “Aprobado”
Si no
mostrar “Suspenso”

- En C

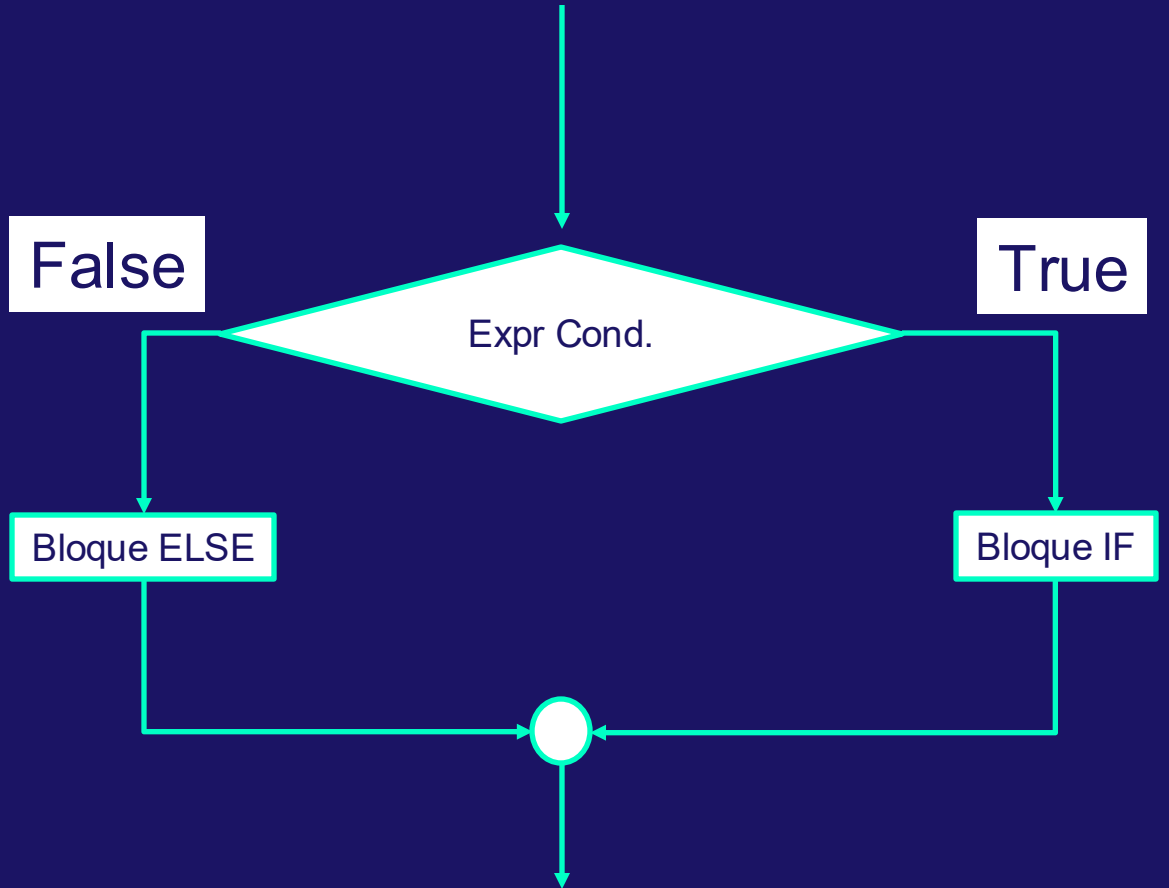
```c
if (nota >= 5) {
    printf("Aprobado\n");
} else {
    printf("Suspenso\n");
}
```

La forma general es:

```
if( <condicion> ){
  < bloque if >;
} else{
  < bloque else >;
}
```

Estructura condicional doble

- El diagrama de flujo asociado es:

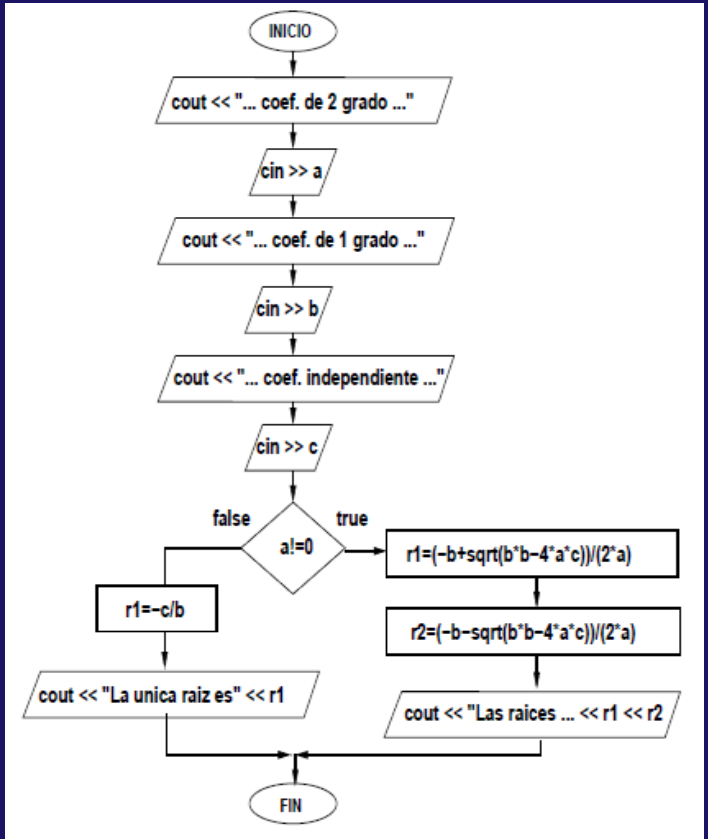


Cálculo de raíces

- En la ecuación de segundo grado, cuando **a==0**, entonces la raíz es única y vale **-c/b**

```
if (a != 0) {
    r1 = ( -b + sqrt( b*b - 4*a*c ) ) / (2*a);
    r2 = ( -b - sqrt( b*b - 4*a*c ) ) / (2*a);
    printf("Las raíces son %lf y %lf\n", r1, r2);
} else {
    r1 = -c / b;
    printf("La única raíz es: %lf\n", r1);
}
```

Cálculo de raíces



¿Cuándo se ejecuta cada instrucción?

```
if (condic_1){
    inst_1;
    if (condic_2)  {
        inst_2;
    }else{
        inst_3;
    }
    inst_4;
}else {
    inst_5;
}
```

	condic_1	***condic_2***
inst_1	*true*	*independiente*
inst_2	*true*	*true*
inst_3	*true*	*false*
inst_4	*true*	*independiente*
inst_5	*false*	*independiente*

Ejercicio doble condición

Suponga que ahorros y gastos son variables de tipo double a las que les ha dado valores.
Escriba una instrucción if-else que realice lo siguiente:

- Si ahorros es mayor o igual que gastos, muestre “SOLVENTE”, reste al valor de ahorros el valor de gastos, y asigne cero a gastos.
- En otro caso debe mostrar “EN QUIEBRA” y no debe modificar el valor de ninguna variable.

Anidamiento de estructuras condicionales

- Transformar una nota numérica en su equivalente en calificación.

```
Si la nota es mayor o igual que 9
  imprimir "Sobresaliente”
  else
   Si la nota es mayor o igual que 7
       imprimir "Notable"
    else
    Si la nota es mayor o igual que 5
         imprimir "Aprobado"
       else
          imprimir "Suspenso“
```

```c
if (nota >= 9)
    printf("Sobre\n");
else if (nota >= 7)
    printf("Notable\n");
else if (nota >= 5)
    printf("Aprobado\n");
else
    printf("Suspenso\n");
```

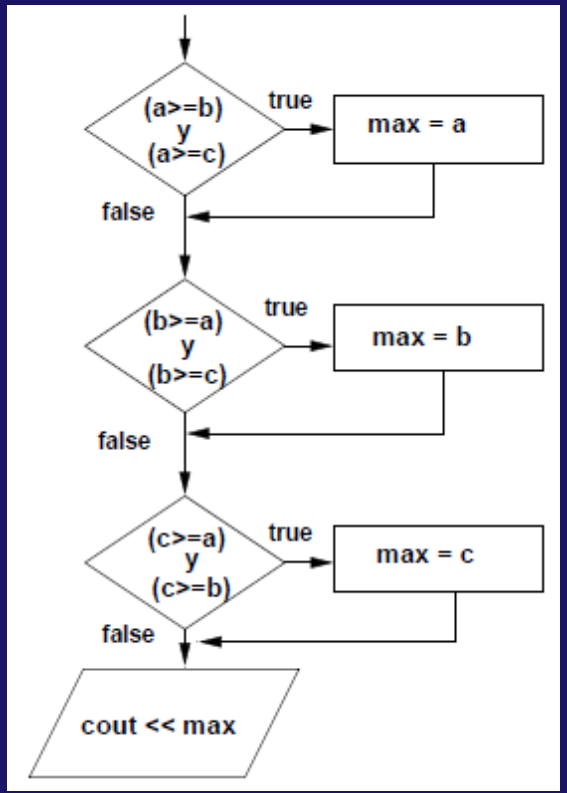
Variabilidad de soluciones

- Dados tres valores a, b y c se requiere calcular el máximo de los 3.
- Veremos que el problema se puede resolver de diferentes maneras.
- Nótese la diferencia entre soluciones.

El mayor de 3 valores (I)

```c
if ((a >= b) && (a >= c))
    max = a;
if ((b >= a) && (b >= c))
    max = b;
if ((c >= a) && (c >= b))
    max = c;

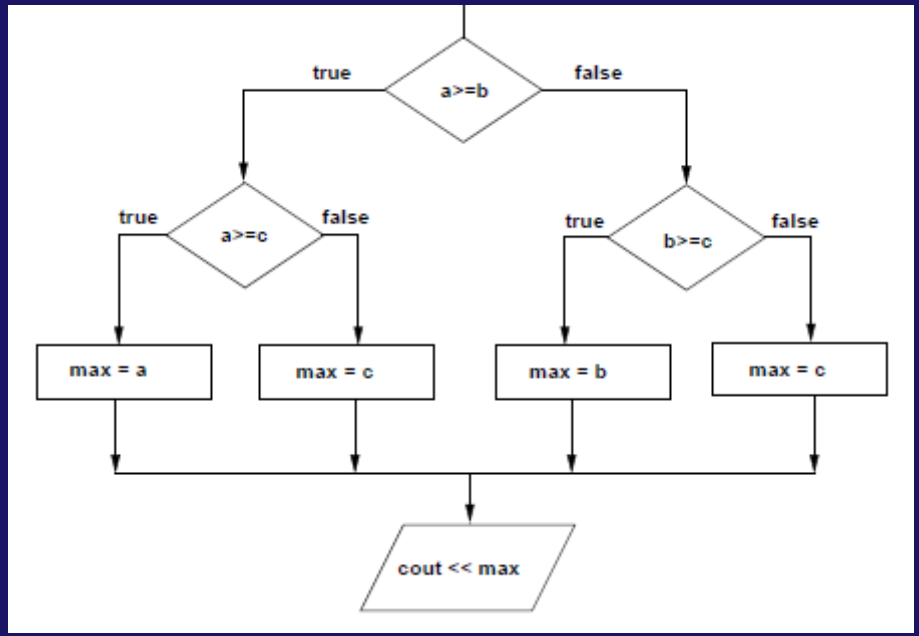
printf("El mayor es %d\n", max);
```



El mayor de 3 valores (II)

```
if (a>=b)
    if (a>=c)
        max = a;
    else
        max = c;
else
    if (b>=c)
        max = b;
    else
        max = c;

printf("El mayor es %d\n", max);
```

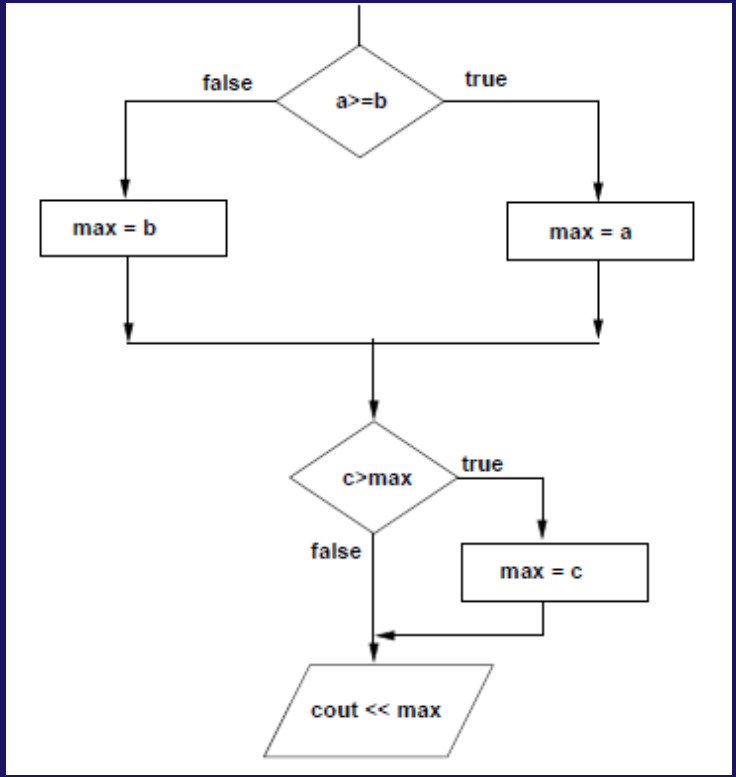


El mayor de 3 valores (III)

```c
if (a >= b)
    max = a;
else
    max = b;

if (c > max)
    max = c;

printf("El mayor es %d\n", max);
```



Condicional múltiple

- Permite definir en una sola estructura, una serie de pares (condición, acción), con condiciones mutuamente excluyentes.
- Muy utilizada en la construcción de menús.

```
switch (<expresión>){
case <constante1>:
        <sentencias1>
        break;
case <constante2>:
        <sentencias2>
        break;
.........................
[default:
        <sentencias>]
}
```

Estructura condicional múltiple

- **<expresión>** es una expresión entera.
- <constante1> o <constante2> es un literal tipo entero o tipo carácter.
- *switch* sólo comprueba la igualdad de la expresión y cada constante.
- No debe haber dos casos (**case**) con la misma *<constante>* en el mismo **switch**. Si esto ocurre, sólo se ejecutan las sentencias correspondientes al caso que aparezca primero.
- El identificador especial **default** permite incluir un caso por defecto, que se ejecutará si no se cumple ningún otro. Se suele colocar como el último de los casos.
- En las estructuras condicionales múltiples también se permite el anidamiento.

Ejemplo 1

```c
int main(){
    int dato1, dato2;
    char opcion;
    printf("Introduce el primer operando: ");
    scanf("%d", &dato1);
    printf("Introduce el segundo operando: ");
    scanf("%d", &dato2);
    printf("Selecciona (S) sumar, (R) restar: ");
    scanf(" %c", &opcion);

    switch (opcion){
        case 'S':
            printf("Suma = %d\n", dato1 + dato2);
            break;
        case 'R':
            printf("Resta = %d\n", dato1 - dato2);
            break;
        default:
            printf("Ninguna operación\n");
    }
    return 0;
}
```

Ejemplo 2

```c
int main(){
    int dato1, dato2;
    char opcion;
    printf("Introduce el primer operando: ");
    scanf("%d", &dato1);
    printf("Introduce el segundo operando: ");
    scanf("%d", &dato2);
    printf("Selecciona (S) sumar, (R) restar: ");
    scanf(" %c", &opcion);

    switch (opcion) {
        case 's':
        case 'S':
            printf("Suma = %d\n", dato1 + dato2);
            break;
        case 'r':
        case 'R':
            printf("Resta = %d\n", dato1 - dato2);
            break;
        default:
            printf("Ninguna operación\n");
    }
}
```

Separando E/S del cómputo

```
/* Rellenamos dato1, dato2 y opción */
bool datos_ok = true;

switch (opcion){
    case 's':
    case 'S':
        rta = dato1 + dato2;
        break;
    case 'r':
    case 'R':
        rta = dato1 - dato2;
        break;
    default:
        datos_ok = false;
}

if (datos_ok)
    printf("%d\n", rta);
else
    printf("Ninguna operacion\n");
```

Cálculo de frecuencias con switch

```
int dado;
int c1, c2, c3, …, c6;
c1 = 0; c2 = 0;….; c6 = 0;
dado = tirarDado();

switch (dado){
 case 1: c1 = c1 + 1; break;
 case 2: c2 = c2 + 1; break;
 case 3: c3 = c3 + 1; break;
    . . .
 case 6: c6 = c6 + 1; break;

}
```

Profundizar en los apuntes

- Separación de cálculos y E/S.
- Evitar repetición de código.
- Ejemplos.
- Comparación entre valores reales.

Ejercicio

Escribe un programa que lea dos enteros desde teclado.

- Si alguno no está entre 0 y 40 (inclusive) debe mostrar “FUERA DE RANGO”,
- En caso contrario debe clasificarlos

1. de TIPO A, si están entre 0 y 10,
2. de TIPO B, si están entre 11 y 20,
3. de TIPO C, si están entre 21 y 30,
4. y de TIPO D, si están entre 31 y 40.

- Además, si están dentro del rango, debe mostrar la media (real) de ambos. (suponemos que los dos valores son del mismo tipo).

02

Estructuras repetitivas

Estructuras repetitivas

Las estructuras repetitivas son también conocidas como *bucles, ciclos o lazos.*

Una estructura repetitiva permite la ejecución de un conjunto de sentencias:

- Mientras se satisface una determinada condición (controladas por condición)
- un número determinado de veces (controladas por contador)

Bucles controlados por condición

- Se ejecuta el conjunto de sentencias de interés mientras la condición sea verdadera.

Pre - Test

```
while (< condición>){
   <cuerpo del bucle>
}
```

Primero se pregunta y luego se ejecuta

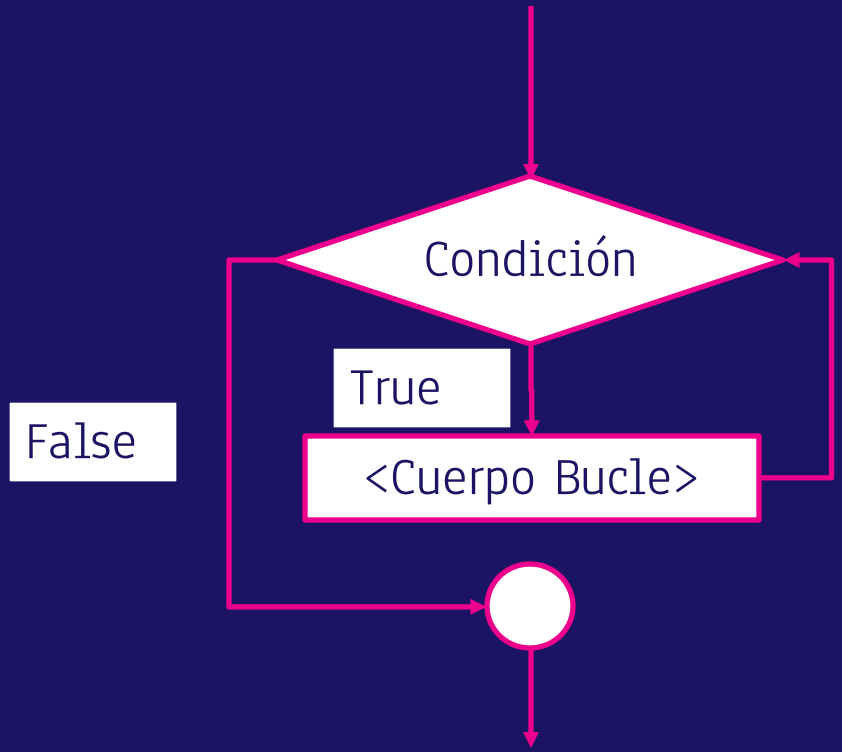
Post - Test

```
do{
   <cuerpo del bucle>
}while (<condición>);
```

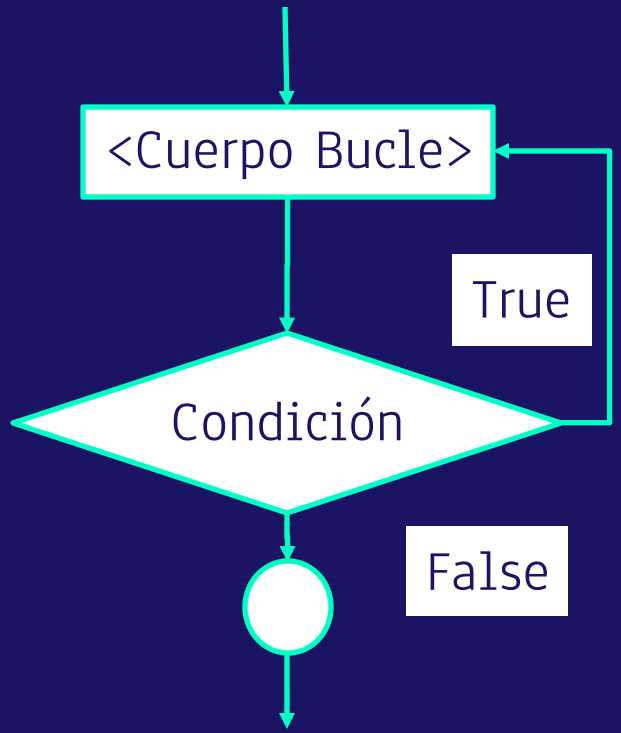
Primero se ejecuta luego se pregunta

Bucles (pre/post) test

Pre - Test



Post - Test



Escribir 10 líneas con 4 estrellas cada una

```
cont = 1;

do {
 printf("****\n");
 cont = cont + 1;
}while (cont <= 10)
```

```
cont = 1;

while (cont <= 10){
 printf("****\n");
 cont = cont + 1;
}
```

```
cont = 0;

do {
 printf("****\n");
 cont = cont + 1;
}while (cont < 10)
```

```
cont = 0;

while (cont < 10){
 printf("****\n");
 cont = cont + 1;
}
```

Escribir 10 líneas con 4 estrellas cada una

```c
scanf("%d", &tope);
cont = 0;

do {
 printf("****\n");
 cont = cont + 1;
} while (cont < tope)
```

Problema si *tope = 0*

Solución

```c
scanf("%d", &tope);
cont = 0;

while (cont < tope){
 printf("****\n");
 cont = cont + 1;
}
```

Ejemplo

- Escribir los números pares entre [0..tope]

```c
scanf("%d", &tope);
i = 0;
while (i <= tope){
  printf("%d\n", i);
  i = i + 2;
}
```

- ¿Qué ocurre si el código anterior se cambia por?

```c
scanf("%d", &tope);
i = 0;
while (i <= tope){
   i = i + 2;
   printf("%d\n", i);
}
```

¿Qué hace este programa?

```c
int main(){
  int i, j;

  i = 0;
  while(i < 4) {
    j = 0;
    while (j < i){
      printf("%d + %d = %d\n", i, j, i + j);
      j = j + 1;
    }
    i = i + 1;
  }
  return 0;
}
```

Ejemplo: control por centinela

- Sumar valores que se leen por teclado hasta que se lee -1. (el valor utilizado para detener el procesamiento se conoce como centinela)

```
suma = 0;
do{
 scanf("%d", &valor);
 suma = suma + valor;
}while (valor != -1)
printf("La suma es %d", suma);
```

Se procesa el valor centinela.
Esta clase de problemas se evita utilizando la técnica de *lectura anticipada*.

Lectura anticipada: se usa un bucle pre-test y se comprueba el primer valor.

```
suma = 0;
scanf("%d", &valor);
while (valor != -1){
   suma = suma + valor;
 scanf("%d", &valor);
}
printf("La suma es %d", suma);
```

Modelización con variables lógicas

```
suma = 0;
do {
    printf("Introduce un valor (-1 para terminar): ");
    scanf("%d", &valor);
    seguir_leyendo = (valor != -1);
    if (seguir_leyendo)
        suma = suma + valor;
} while (seguir_leyendo);

printf("La suma es %d\n", suma);
```

Para no repetir código en los bucles que leen datos, o bien usaremos la técnica de lectura anticipada o bien introduciremos variables lógicas que controlen la condición de terminación de lectura

Filtrado de entrada con ciclos post-test

- Los ciclos post-test son útiles para forzar que los datos de entrada pertenezcan a un rango o conjunto de opciones predeterminados.

```c
/* filtro para valores positivos */
do {
    printf("Introduzca un valor > 0: ");
    scanf("%d", &valor);
} while (valor <= 0);
```

```c
/* filtro para 's','S','n','N' */
do {
    printf("Desea Salir (s/n)?: ");
    scanf(" %c", &opc);
} while ((opc != 's') && (opc != 'S') && (opc != 'n') && (opc != 'N'));
```

Ejercicios

1. Hacer un programa que muestre los pares de valores siguientes, utilizando una sucesión: (1,10), (2,9), (3,8)…
2. Hacer un programa que muestre las ternas (x,y,z) donde x,y,z [1..5]. Ej: (1, 1, 1); (1, 1, 2); … ; (5, 5, 5).
3. Tirar N veces un dado y contar las ocurrencias de cada valor. Suponga que existe una función `tirarDado()` que devuelve un número entero entre 1 y 6.

Resolución de un problema

- Tenemos que realizar un programa que calcule la nota promedio de un conjunto de notas.
- La cantidad de notas puede variar en cada ejecución del programa.
  - Utilizaremos un valor "centinela" para indicar el fin de las notas.
  - El proceso terminará cuando se ingrese dicho valor.
  - El valor concreto del centinela debe ser diferente al de cualquier posible entrada válida (en este caso podría ser -1)

Algoritmo y refinamientos

Nuestro problema es:

Determinar el promedio de las notas de un examen

Esta tarea la podemos subdividir en otras más pequeñas:

1. *Inicializar variables*
2. *Ingresar, Sumar y Contar las notas del examen*
3. *Calcular e imprimir el promedio*

Algoritmo y refinamientos

La fase de inicialización *Inicializar variables* se puede refinar a:

```
Inicializar la variable total a cero
Inicializar el contador de notas a cero
```

La fase Ingresar, Sumar y Contar las notas del examen, se puede refinar a:

```
Ingresar la primera nota (quizá centinela)
Mientras el usuario no ingrese el valor centinela
    sumar la nota al total
    sumar uno al contador
    Ingresar la siguiente nota (quizá centinela)
```

Algoritmo y refinamientos

La fase Calcular e imprimir el promedio se puede refinar a

```
Si la cantidad de notas es distinta de cero
   el promedio es la suma de las notas dividido la
   cantidad
   Imprimir el promedio
En caso contrario
   Imprimir “No se ingresaron notas"
```

Algoritmo final

Inicializar la variable total a cero
Inicializar el contador de notas a cero
Ingresar la primera nota (posiblemente el centinela)

Mientras el usuario no ingrese el valor centinela
    sumar la nota al total
    sumar uno al contador
    Ingresar la siguiente nota (el centinela ??)

Si la cantidad de notas es distinta de cero
    el promedio es la suma de las notas dividido la cantidad
    Imprimir el promedio
En caso contrario
    Imprimir “No se ingresaron notas"

Resolución C

```c
int main() {
    const double CENTINELA = -1.0;
    double contador, suma, avg, nota;
    suma = contador = 0;

    printf("Ingrese la nota (-1 para fin): ");
    scanf("%lf", &nota);  /* Se utiliza %lf para leer un valor tipo double */

    while (nota != CENTINELA) {
        suma = suma + nota;
        contador = contador + 1;
        printf("Ingrese la nota (-1 para fin): ");
        scanf("%lf", &nota);
    }

    if (contador > 0) {
        avg = suma / contador;
        printf("El promedio es %.2lf\n", avg);  /* %.2lf para mostrar dos decimales */
    } else {
        printf("No se ingresaron notas\n");
    }

    return 0;
}
```

Extensiones (trabajo autónomo)

1. Extender el programa anterior para que informe cuál fue la nota más alta y cuál la más baja.
2. Chequear que las notas estén en el rango correcto [0,10]
3. Suponga que se leen valores enteros desde teclado.
4. El procesamiento termina cuando se lee un cero.
5. Informar la media de los valores positivos (aprobados) y la media de los negativos (suspensos).

Modificación de variables

```
/*Ir dividiendo n por 10 hasta llegar a una cifra
El numero de digitos sera el numero de iteraciones  */

num_digitos = 1;
while (n > 9){
  n = n/10;
  num_digitos++;
}
```

```
/* Mejor si no modificamos la variable original:  */
n_dividido = n;
num_digitos = 1;
while (n_dividido > 9){
  n_dividido = n_dividido/10;
  num_digitos++;
}
```

¿Qué hace este código?

```c
int main() {
  int i, resto;

  printf("Ingrese un número: ");
  scanf("%d", &i);

  while (i > 0) {
    resto = i % 10;
    i = i / 10;
    printf("%d\n", resto);
  }

  return 0;
}
```

Condiciones compuestas

- El uso de condiciones compuestas es común para establecer los límites en las instrucciones de bucles.
- Ejemplo → Calcular el M.C.D. de dos números:
  - Primer posible divisor: el menor de ambos
  - Mientras divisor no divida a ambos: probar con el anterior

```
if (b < a){
  menor = b;
}else{
  menor = a;
}
divisor = menor;
while (a % divisor != 0 || b % divisor != 0){
  divisor --;
}
max_com_div = divisor;
```

Bucles controlados por contador

- Hemos comprobado en diversos ejercicios con el bucle `while`, que en muchos casos se repite un mismo esquema:
  1. Hay que hacer una inicialización antes del bucle.
  2. El bucle comprueba únicamente una condición de tipo “mientras ”.
  3. Al final del bucle hay que actualizar una variable para preparar la siguiente iteración.

- El bucle `for` nos permite escribir código más fácil de leer y mantener al unir todos los elementos que indican la lógica de control del bucle en una misma línea.

Bucle for

- Por tanto, si conocemos exactamente la cantidad de veces que necesitamos repetir un conjunto de sentencias DEBEMOS usar un bucle `for`.
- En general, los bucles controlados por contador requieren
  - Una variable de control o contador
  - Un valor inicial para el contador
  - Un valor final para el contador
  - Una condición para verificar si la variable de control alcanzó su valor final.
  - Un valor de incremento (o decremento) con el cual se modifica la variable de control en cada bucle

Bucle for

- La forma general del bucle for es

```
for (inicializacion; condicion continuación; incremento ){
   < conjunto de sentencias>
}
```

- Ejemplo: imprimir los valores enteros entre 1 y 10

```
for( nro = 1; nro <= 10; nro = nro + 1){
  printf("%d\n", nro);
}
```

Nunca cambiaremos el valor de la variable de control dentro del bucle

Relación entre for y while

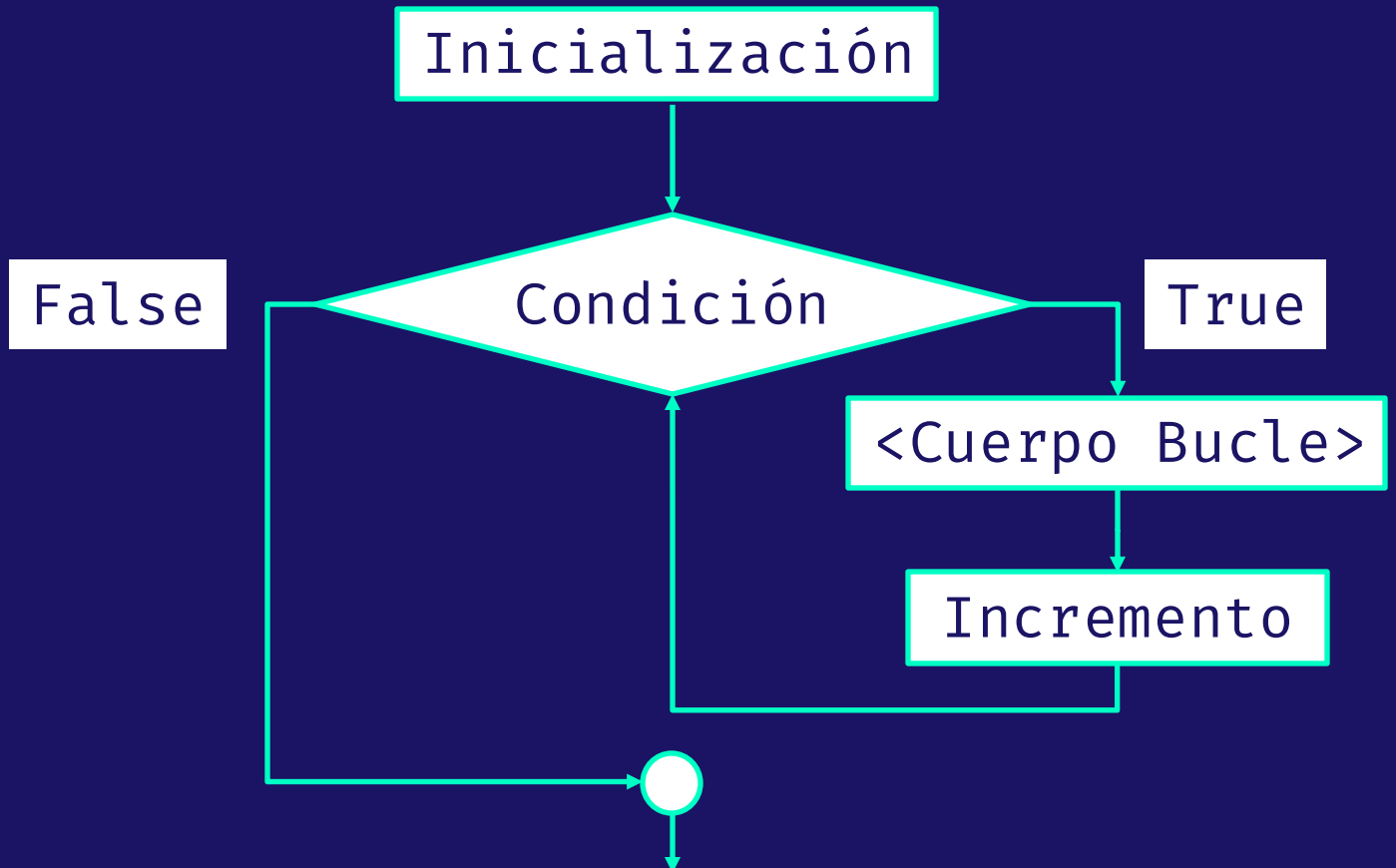
- Se debe notar que los bucles FOR se pueden reescribir como bucles WHILE

```
for ( inicialización; condición continuación; incremento )
   < conjunto de sentencias>
```



```
inicialización
while (condición continuación){
  < conjunto de sentencias>
  incremento
}
```

Diagrama de flujo for



Errores comunes

- Error en divisores de un número.

```c
int numero;
int divisor = 1;

printf("Introduzca número positivo: ");
scanf("%d", &numero);

printf("Los divisores son:\n");
while (divisor < numero) {
    if (numero % divisor == 0)
        printf("Divisor: %d\n", divisor);
    else
        divisor++;
}
```

Errores comunes

- Error en listado de los pares menores.

```c
int numero, i;

printf("Introduzca número positivo: ");
scanf("%d", &numero);

printf("Los pares menores son: ");
for (i = 2; i != numero; i += 2)
    printf("%d ", i);
```

- Error en cuenta regresiva.

```c
unsigned int i;

printf("¡Cuenta regresiva!\n");
for (i = 10; i >= 0; i--)
    printf("...%d", i);
```

Errores comunes

- Error en media de 10 números.

```c
double suma = 0, dato; int i;
for (i = 0; i < 10; i--) {
    printf("Introduce dato: ");
    scanf("%lf", &dato);
    suma += dato;
}
printf("Media: %lf\n", suma / 10);
```

- Error en tablas de multiplicar.

```c
int i,j;
for (i = 10; i > 0; --i) {
    printf("Tabla del %d:\n", i);
    for (int j = 1; j <= 10; ++j);
        printf("%d x %d = %d\n", i, j, i * j);
}
```

Códigos “mejorables”

- Búsqueda de números primos

```
int numero,i;
bool es_primo = true;

printf("Introduzca número positivo: ");
scanf("%d", &numero);

for (i = 2; i < sqrt(numero); ++i) {
    if (numero % i == 0)
        es_primo = false;
}
```

- Buscar primer divisor:

```
int numero,i;
int divisor = 0;

printf("Introduzca número positivo: ");
scanf("%d", &numero);

for (i = 2; i < numero; ++i) {
    if (numero % i == 0) {
        divisor = i;
        i = numero;
    }
}

if (divisor != 0)
    printf("Localizado: %d\n", divisor);
```

Ejercicios

1. Hacer un programa que lea 10 valores enteros y muestre cual ha sido el máximo y cual el mínimo.
2. Generar todas los pares (x,y) con x,y en [1,10] tales que x < y
3. Implemente un programa que, dado un valor n, muestre n líneas, de forma tal que la línea i, contenga i asteriscos (*).
4. Modifique el programa anterior, para que la línea i, tenga n-i asteriscos.
5. Rellena la tabla de multiplicar de los TOPE primeros números. Rellena la tabla con dos topes: izquierda y derecha.

04

Estructuras perniciosas

Otras estructuras de control

- Existen más estructuras de control que permiten modificar el flujo:
  - goto
  - continue
  - break
  - exit
- Hace más de 60 años que se demostró que su uso era la principal razón de problemas del desarrollo de software
  - Especialmente peligrosa es goto
- No permitiremos el uso de ninguna de estas estructuras (salvo break en el switch) → SUSPENSO si se usan